UNIVERSIDADE ABERTA

Formas Normais: da Lógica Matemática à Computação

Olivia Pachele Mattiazzo

Mestrado em Estatística, Matemática e Computação

Orientadores: Prof.ª Dr.ª Gilda Ferreira
e Prof.ª Dr.ª Yves Robert

Novembro 2026

UNIVERSIDADE ABERTA

Formas Normais: da Lógica Matemática à Computação

Olivia Pachele Mattiazzo

Mestrado em Estatística, Matemática e Computação

Novembro 2026

Agradecimentos

Resumo

Palavras-chave:

Abstract

Keywords:

Resumo Alargado em Português

Índice

Introdução **27**

1 Contexto Histórico e Fundamentos Teóricos **29**
1.1 Origens das formas normais na lógica 29
1.2 Desenvolvimento e importância na computação 30
1.3 Definições e propriedades: negativa, conjuntiva, disjuntiva e prenexa 31
1.4 Completude funcional: bases de conectivos 34
1.5 Limitações teóricas 35

2 Algoritmos de Transformação **39**
2.1 Revisão dos principais métodos de conversão para formas normais 39
2.2 Correção, equivalência lógica e complexidade computacional 41
2.3 Considerações sobre implementação computacional 46

3 Projeto e Implementação da Ferramenta **50**
3.1 Requisitos funcionais e arquitetura da solução 50
3.2 Módulos de transformação algorítmica 50
3.3 Interface pedagógica 50
3.4 Validação: testes automatizados 50

4 Conclusão **51**
4.1 Síntese dos resultados obtidos 51
4.2 Contribuições para o ensino de lógica 51
4.3 Limitações do trabalho realizado 51
4.4 Perspetivas de evolução e trabalhos futuros 51

References **52**

A Algoritmos de Referência em Pseudocódigo **55**
A.1 Conversão para a forma normal negativa 55
A.2 Conversão para a forma normal conjuntiva e disjuntiva 56

A.3 Simplificações que preservam a equivalência 56
A.4 Procedimento completo 58

B Scripts de Transformação **59**
B.1 Scripts de Transformação 59

Lista de Tabelas

1.1 Comparação das quatro formas normais discutidas na Seção 1.3 33

2.1 Síntese comparativa dos métodos de conversão analisados na Seção 2.2 . . 49

Lista de Figuras

1.1 Linha do tempo (fora de escala) dos principais marcos históricos discutidos no Capítulo 1 35
1.2 Rotas de conversão de uma fórmula arbitrária para formas normais, com a alternativa de Tseitin 37

Lista de Abreviações e Acrónimos

CNF Conjunctive Normal Form (Forma Normal Conjuntiva)

coNP Complemento de NP (classe de complexidade)

DNF Disjunctive Normal Form (Forma Normal Disjuntiva)

DPLL Davis–Putnam–Logemann–Loveland (algoritmo)

NAND Negação Conjunta (*Not AND*; traço de Sheffer)

NNF Negation Normal Form (Forma Normal Negativa)

NOR Negação Disjunta (*Not OR*; seta de Peirce)

NP Nondeterministic Polynomial time (classe de complexidade)

P Polynomial time (classe de complexidade)

PNF Prenex Normal Form (Forma Normal Prenexa)

SAT Boolean Satisfiability Problem (Problema de Satisfazibilidade Booleana)

Introdução

Apesar de não vermos explicitamente os conceitos matemáticos mais complexos em ação quando usamos uma interface de um sistema operacional ou quando jogamos videogame, por exemplo, eles estão lá por debaixo dos panos. Não é possível imaginar aprimoramento e refinamento de algoritmos para uma melhor perfomance sem levar em conta os pilares matemáticos que estão por trás do digital. Tudo o que temos em termos de tecnologia, desde as mais básicas como linguagem de máquina, até as mais avançadas, como as polêmicas Inteligências Artificiais generativas, tem seus pilares sustentados em conceitos matemáticos desenvolvidos a base de muito papel e caneta. Podem parecer conceitos antigos e/ou ultrapassados mas, sem eles, não teríamos estrutura para o passo a frente em termos de evolução tecnológica.

Walicki, em *Introduction to Mathematical Logic* (2011) [17], considera que o conceito de lógica foi inventado por Parmenides, no Egito, pois foi o primeiro filósofo que não somente propôs uma visão da realidade mas também a apoiou em uma extensa argumentação. Passando por muitos filósofos e matemáticos da antiguidade, chegamos eventualmente no período entre a segunda metade do século XIX e meados do século XX, onde os sistemas e cálculo de Leibniz marcam a aparição de uma linguagem formalizada e simbólica mais alinhada com a manipulação matemática. A partir disso, passamos por Geoge Boole, Gottlob Frege, Russell e Whitehead, que serão mais aprofundados conforme esse trabalho avança, mas que desenvolveram, formalizaram e consolidaram a manipulação formal de fórmulas. No cenário desenvolvido por estes mestres, as formas normais passaram a desempenhar um papel central na análise da dedução.

Um dos desafios de hoje, no entanto, se dá no ensino da lógica matemática e computacional aos novos alunos dos cursos de exatas. Existem ferramentas que auxiliam no entendimento, como ambientes de programação visual e ferramentas para aprendizado de lógica de programação, como, por exemplo, o *Scratch, Inventor, AgentSheets, RoboMind*, etc. [6] [16] Porém, existem dois principais obstáculos no uso dessas ferramentas: primeiro que tem uma abordagem mais infantil do ensino, e segundo, estão mais focados na lógica computacional e de programação do que no ensino de lógica matemática em si. Há uma lacuna que este trabalho visa preencher em termos de ferramenta pedagógica para auxiliar no aprendizado de formas normais em um contexto mais acadêmico.

O objetivo geral desta dissertação é examinar as formas normais no âmbito da lógica matemática, situando seu surgimento no contexto histórico da formalização da matemática entre o final do século XIX e o início do século XX. Busca-se compreender como a sistematização das linguagens formais, especialmente a partir dos trabalhos de Boole, Frege, Russell e seus sucessores, contribuiu para a consolidação de procedimentos de transformação de fórmulas, culminando na definição e no uso das formas normais. Além da análise histórica e conceitual, pretende-se investigar a relevância dessas estruturas na computação contemporânea. Por fim, o trabalho propõe discutir as principais dificuldades enfrentadas por estudantes no processo de aprendizagem da lógica matemática e computacional.

Como desdobramento dos objetivos gerais, esta dissertação tem como objetivo específico conceber e implementar uma ferramenta pedagógica destinada a apoiar a compreensão das transformações envolvidas na obtenção das formas normais conjuntiva e disjuntiva. Para além da construção do sistema, o trabalho registra e analisa as decisões técnicas que fundamentaram sua arquitetura e funcionamento, bem como os procedimentos de teste adotados, buscando assegurar consistência, correção lógica e adequação didática. Dessa forma, a ferramenta não se configura apenas como um produto computacional, mas também como um artefato de investigação no âmbito do ensino de lógica.

Capítulo 1

Contexto Histórico e Fundamentos Teóricos

1.1 Origens das formas normais na lógica

Nos finais do século XIX e início do século XX, a matemática começou a passar por uma transformação significativa, com ênfase crescente na rigorosidade e no método axiomático, estabelecendo as bases para a lógica matemática moderna. A chamada Lógica Simbólica Moderna foi desenvolvida, em parte, como uma maneira de prover uma estrutura formal para a manipulação de ideias e argumentos matemáticos. Depois de alguns avanços, também ganhou o objetivo de reduzir expressões lógicas a padrões mais simples que preservassem a equivalência lógica das demonstrações formais. A lógica também influenciou a ciência da computação, especialmente em áreas como raciocínio automatizado, verificação de programas e sistemas, e sistemas de tipos para linguagens de programação. Exemplos incluem o uso de álgebra booleana em circuitos de comutação, algoritmos para prova automática de teoremas e o desenvolvimento de linguagens de programação tipadas. [18]

Tradicionalmente, a história da lógica moderna é narrada como a história da lógica matemática ou simbólica, que foi criada por matemáticos. George Boole é apontado como um dos principais pioneiros dessa área, com sua obra "The Mathematical Analysis of Logic"(1847), que marcou o início do desenvolvimento contínuo da lógica matemática. Boole introduziu a ideia de representar proposições lógicas por meio de símbolos e operações matemáticas, criando o que ele chamou de "Cálculo da Lógica". Ele buscava estabelecer uma base psicológica para a lógica, mediada pela linguagem, e propôs que a lógica deveria ser associada à matemática, e não à metafísica. Sua abordagem inovadora permitiu que proposições lógicas fossem tratadas como teoremas matemáticos, utilizando leis de combinação semelhantes às da álgebra simbólica.

A lógica matemática de Boole foi inicialmente ignorada por muitos matemáticos e

filósofos, devido à falta de tradição em lógica formal na Grã-Bretanha e à visão predominante na Alemanha de que a lógica formal já era um campo completo desde Aristóteles. No entanto, após sua morte em 1864, o trabalho de Boole começou a ganhar reconhecimento, especialmente por meio de William Stanley Jevons, que simplificou e popularizou o cálculo lógico de Boole, e Alexander Bain, que introduziu as contribuições de Boole em sua obra sobre lógica. A lógica matemática de Boole foi fundamental para o desenvolvimento da lógica simbólica como uma subdisciplina da matemática no início do século XX. Sua abordagem abriu caminho para aplicações em diversas áreas, como fundamentos da matemática, física, teoria das probabilidades e filosofia da ciência. [8]

1.2 Desenvolvimento e importância na computação

Se a seção anterior situou o surgimento das formas normais no interior do próprio desenvolvimento da lógica simbólica, cabe agora examinar como esse aparato conceitual migrou da matemática pura para o domínio da computação, tornando-se um dos seus pilares operacionais. O ponto de inflexão costuma ser atribuído a Claude Shannon que, em sua dissertação de mestrado *A Symbolic Analysis of Relay and Switching Circuits* (1938), demonstrou que a álgebra booleana desenvolvida por Boole podia servir de base para a análise e a síntese de circuitos de relés e comutação elétrica [13]. Ao representar estados de circuito por meio de variáveis binárias e operações lógicas, Shannon evidenciou que problemas de engenharia elétrica podiam ser tratados como problemas puramente algébricos, e vice-versa: circuitos podiam ser projetados a partir da manipulação simbólica de expressões booleanas. Essa correspondência tornou as formas normais, sobretudo a conjuntiva e a disjuntiva, instrumentos naturais de trabalho, uma vez que qualquer função booleana pode ser expressa de modo sistemático como uma disjunção de conjunções (soma de produtos) ou como uma conjunção de disjunções (produto de somas), fornecendo assim um procedimento uniforme para simplificação e implementação de circuitos lógicos.

A relevância computacional das formas normais se aprofunda ainda mais com o advento do raciocínio automatizado. Em 1960, Martin Davis e Hilary Putnam propuseram um procedimento para testar a satisfazibilidade de fórmulas da lógica de primeira ordem que tomava como entrada expressões já convertidas para a forma normal conjuntiva, tratando cada fórmula como uma conjunção de cláusulas disjuntivas de literais [5]. O procedimento de Davis e Putnam, posteriormente refinado por Davis, Logemann e Loveland no algoritmo hoje conhecido como DPLL, consolidou a forma normal conjuntiva como o formato padrão de entrada para os problemas de satisfazibilidade booleana (SAT), estabelecendo um paradigma que permanece no núcleo dos solucionadores de SAT contemporâneos, empregados em verificação de hardware e software, planejamento automatizado e otimização combinatória.

A adoção da forma normal conjuntiva como representação canônica não é arbitrária: ela decorre de propriedades estruturais que facilitam a aplicação de regras de inferência, como a resolução, de modo mecânico e sistemático. Uma fórmula em CNF permite que a verificação de sua satisfazibilidade seja reduzida à manipulação de conjuntos de cláusulas, o que se presta naturalmente à implementação em algoritmos de busca e retrocesso (*backtracking*). Por outro lado, a forma normal disjuntiva, embora menos utilizada em provadores automáticos de teoremas por razões de eficiência computacional, permanece central em áreas como minimização de circuitos digitais, síntese lógica e sistemas de regras em inteligência artificial, nos quais a formulação de uma função como disjunção de termos conjuntivos corresponde diretamente à enumeração de condições suficientes para a verdade de uma proposição.

Cabe ainda destacar que a conversão de uma fórmula arbitrária para forma normal, seja conjuntiva, seja disjuntiva, não é um processo computacionalmente trivial: no pior caso, o tamanho da fórmula resultante pode crescer exponencialmente em relação ao tamanho da fórmula original, o que impõe limites práticos à aplicação ingênua desses procedimentos em sistemas de larga escala. Essa tensão entre a elegância teórica das formas normais e o custo computacional de sua obtenção é um dos fios condutores que atravessam esta dissertação, e será retomada com maior profundidade no Capítulo 2, dedicado aos algoritmos de transformação e à sua análise de correção e complexidade.

1.3 Definições e propriedades: negativa, conjuntiva, disjuntiva e prenexa

Antes de avançar para os algoritmos de conversão propriamente ditos, no Capítulo 2, é necessário fixar com precisão o que se entende por uma *forma normal* e apresentar as quatro formas que estruturam esta dissertação: a forma normal negativa, a forma normal conjuntiva, a forma normal disjuntiva e a forma normal prenexa. Em termos gerais, uma forma normal é uma representação canônica de uma fórmula lógica que obedece a restrições sintáticas específicas quanto à disposição dos conectivos e, no caso da lógica de primeira ordem, dos quantificadores, preservando a equivalência lógica com a fórmula original [17]. Cada uma das quatro formas a seguir impõe um conjunto distinto dessas restrições, e é justamente essa diversidade de critérios que torna necessário tratá-las separadamente antes de discutir, na Seção 1.4, a completude funcional das bases de conectivos que as sustentam.

A **forma normal negativa** (*Negation Normal Form*, NNF) constitui o primeiro passo desse processo de normalização. Uma fórmula está em NNF quando os únicos conectivos presentes são a conjunção, a disjunção e a negação, e esta última incide exclusivamente

sobre fórmulas atômicas, isto é, quando toda subfórmula negada é um literal [17]. A obtenção da NNF a partir de uma fórmula arbitrária passa por dois procedimentos sistemáticos: primeiro, a eliminação dos conectivos de implicação e de bicondicional, reescritos em termos de conjunção, disjunção e negação; segundo, o deslocamento das negações para o interior da fórmula por meio das leis de De Morgan e da eliminação da dupla negação, até que restem apenas negações de literais. Todo formula da lógica proposicional clássica admite uma NNF equivalente, e essa equivalência é o que garante que os passos seguintes de normalização — a saber, a obtenção das formas conjuntiva e disjuntiva — possam partir de uma base sintaticamente controlada, sem alterar o significado da fórmula original.

A partir de uma fórmula em NNF, aplicam-se as leis distributivas da conjunção sobre a disjunção (ou vice-versa) para se obter as formas conjuntiva e disjuntiva. A **forma normal conjuntiva** (*Conjunctive Normal Form*, CNF) é definida como uma conjunção de cláusulas, em que cada cláusula é, por sua vez, uma disjunção de literais: $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_n$, onde cada $C_i = l_1 \vee l_2 \vee \cdots \vee l_k$ e cada l_j é uma variável proposicional ou sua negação. Simetricamente, a **forma normal disjuntiva** (*Disjunctive Normal Form*, DNF) é definida como uma disjunção de termos, em que cada termo é uma conjunção de literais: $\varphi = D_1 \vee D_2 \vee \cdots \vee D_n$, onde cada $D_i = l_1 \wedge l_2 \wedge \cdots \wedge l_k$. Ambas as formas guardam uma relação de dualidade entre si: a negação de uma fórmula em CNF, reescrita em NNF, resulta em uma fórmula em DNF, e vice-versa. Todo formula da lógica proposicional admite uma CNF e uma DNF equivalentes, o que pode ser demonstrado tanto por indução sobre a estrutura da fórmula a partir da sua NNF quanto de forma construtiva a partir de sua tabela-verdade, tomando a disjunção das conjunções correspondentes às linhas em que a fórmula é verdadeira (para a DNF) ou a conjunção das disjunções correspondentes às linhas em que é falsa (para a CNF) [3]. É importante notar, no entanto, que nem a CNF nem a DNF de uma fórmula são únicas: diferentes sequências de aplicação das leis distributivas e comutativas podem produzir formas conjuntivas ou disjuntivas distintas, ainda que logicamente equivalentes entre si, o que tem implicações diretas para a complexidade computacional discutida na Seção 1.2 e retomada no Capítulo 2.

Por fim, a **forma normal prenexa** (*Prenex Normal Form*, PNF) distingue-se das três anteriores por pertencer ao domínio da lógica de primeira ordem, na qual a presença de quantificadores universais e existenciais exige um tratamento sintático adicional. Uma fórmula está em PNF quando assume a forma $Q_1 x_1 \, Q_2 x_2 \cdots Q_n x_n \, M$, em que cada Q_i é um quantificador ($\forall$ ou $\exists$) e M, chamada de matriz da fórmula, não contém quantificadores. A ideia de reunir todos os quantificadores num prefixo antecedendo uma matriz livre de quantificadores remonta ao trabalho de Charles Sanders Peirce, que em 1885, no artigo *On the Algebra of Logic: A Contribution to the Philosophy of Notation*, introduziu regras para deslocar os quantificadores para a esquerda da fórmula, antecipando o que viria a ser

sistematizado como forma normal prenexa [9]. Assim como ocorre com a NNF, a CNF e a DNF, todo fórmula de primeira ordem é logicamente equivalente a uma fórmula em PNF, obtida por meio de regras de equivalência que permitem mover cada quantificador para fora do escopo dos conectivos proposicionais, com o cuidado de renomear variáveis ligadas sempre que necessário para evitar capturas indevidas. Uma vez em PNF, a matriz M pode ainda ser colocada em CNF ou em DNF, resultando nas chamadas formas normais prenexa conjuntiva e prenexa disjuntiva, respectivamente — uma combinação particularmente relevante para os procedimentos de prova automática de teoremas mencionados na Seção 1.2, uma vez que isola, de um lado, a estrutura quantificacional da fórmula e, de outro, sua estrutura proposicional.

Em conjunto, essas quatro formas normais evidenciam um traço comum: todas preservam a equivalência lógica da fórmula original, mas nenhuma delas é única, e a transformação de uma fórmula arbitrária em qualquer uma delas pode acarretar um custo computacional não trivial, como já apontado na Seção 1.2. Essa tensão entre canonicidade sintática, de um lado, e eficiência de obtenção, de outro, é o que motiva a discussão da Seção 1.4, dedicada à completude funcional das bases de conectivos que tornam essas formas possíveis, e será retomada de maneira mais técnica no Capítulo 2, no qual os algoritmos de conversão e sua complexidade computacional são analisados em detalhe.

A Tabela 1.1 sintetiza as quatro formas normais apresentadas nesta seção, contrastando o domínio lógico em que cada uma se aplica, a restrição sintática que a define e um exemplo típico de fórmula que a satisfaz.

Tabela 1.1: Comparação das quatro formas normais discutidas na Seção 1.3

Forma	**Sigla**	**Domínio**	**Restrição sintática**	**Exemplo típico**
Forma Normal Negativa	NNF	Lógica proposicional	Negação incide apenas sobre literais; únicos conectivos são $\wedge$, $\vee$, $\neg$	$\neg p \wedge (q \vee \neg r)$
Forma Normal Conjuntiva	CNF	Lógica proposicional	Conjunção de cláusulas; cada cláusula é uma disjunção de literais	$(p \vee \neg q) \wedge (\neg p \vee r)$
Forma Normal Disjuntiva	DNF	Lógica proposicional	Disjunção de termos; cada termo é uma conjunção de literais	$(p \wedge q) \vee (\neg p \wedge r)$
Forma Normal Prenexa	PNF	Lógica de primeira ordem	Todos os quantificadores precedem uma matriz livre de quantificadores	$\forall x \, \exists y \, (P(x) \vee Q(x, y))$

1.4 Completude funcional: bases de conectivos

As definições e os teoremas de existência apresentados na Seção 1.3 repousam, ainda que implicitamente, sobre uma pressuposição que convém agora explicitar: a de que os conectivos escolhidos — conjunção, disjunção e negação — são suficientes para expressar qualquer função booleana, e portanto qualquer fórmula da lógica proposicional. Essa suficiência não é uma propriedade trivial nem exclusiva desse conjunto particular de conectivos; trata-se de uma propriedade metateórica conhecida como **completude funcional**, e sua investigação é o que permite compreender por que certas bases de conectivos servem de fundamento para as formas normais, enquanto outras não.

Formalmente, um conjunto de conectivos $\mathcal{B}$ é dito funcionalmente completo quando toda função booleana $f : \{0,1\}^n \to \{0,1\}$, para qualquer n, pode ser expressa por uma fórmula construída exclusivamente a partir dos conectivos de $\mathcal{B}$ e de variáveis proposicionais. O conjunto $\{\wedge, \vee, \neg\}$, empregado nas definições de NNF, CNF e DNF na seção anterior, é funcionalmente completo, mas é também redundante: por força das leis de De Morgan, a disjunção pode ser definida a partir da conjunção e da negação (ou vice-versa), de modo que já os conjuntos $\{\wedge, \neg\}$ e $\{\vee, \neg\}$ bastam para expressar qualquer função booleana. A demonstração da completude funcional de conjuntos como esses decorre diretamente do teorema de existência da forma normal disjuntiva discutido na Seção 1.3: como toda função booleana pode ser lida a partir de sua tabela-verdade e reescrita como uma disjunção das conjunções correspondentes às linhas em que assume valor verdadeiro, qualquer conjunto de conectivos capaz de expressar conjunção, disjunção e negação é, por construção, funcionalmente completo.

O primeiro tratamento sistemático da questão da completude funcional é atribuído a Emil Post que, em sua dissertação de doutoramento de 1920, publicada em 1921 sob o título *Introduction to a General Theory of Elementary Propositions*, não apenas demonstrou a completude do cálculo proposicional de *Principia Mathematica*, de Whitehead e Russell, como também investigou sistematicamente quais subconjuntos de funções booleanas eram fechados sob composição [11]. Esse trabalho deu origem ao que hoje se conhece como reticulado de Post, uma classificação completa de todos os *clones* de funções booleanas — isto é, de todos os conjuntos de funções fechados sob composição e contendo as projeções — da qual decorre, como corolário, um critério necessário e suficiente para a completude funcional de qualquer conjunto de conectivos: um conjunto $\mathcal{B}$ é funcionalmente completo se e somente se não estiver inteiramente contido em nenhuma das cinco classes maximais de funções booleanas identificadas por Post, a saber, as funções que preservam o valor verdadeiro, as que preservam o valor falso, as autoduais, as monótonas e as lineares (afins).

Um resultado particularmente relevante para a computação, e que ilustra de maneira

contundente o alcance da noção de completude funcional, é a existência de conectivos isolados que, sozinhos, já constituem uma base funcionalmente completa. O caso mais conhecido é o da *negação conjunta* (NAND), também chamada de traço de Sheffer, introduzida por Henry Sheffer em 1913 como parte de uma axiomatização alternativa da álgebra booleana [14]. A partir do NAND isoladamente é possível definir a negação ($p \uparrow p$), a conjunção ($(p \uparrow q) \uparrow (p \uparrow q)$) e, por extensão, todos os demais conectivos booleanos, o que faz do NAND — e, dualmente, do NOR, por vezes chamado de seta de Peirce — o único conectivo binário necessário para expressar qualquer fórmula proposicional. Essa propriedade não é apenas uma curiosidade lógica: é o fundamento teórico que permite que circuitos digitais inteiros sejam implementados unicamente a partir de portas NAND ou unicamente a partir de portas NOR, uma técnica amplamente empregada em projeto de circuitos integrados, e que retoma diretamente a discussão da Seção 1.2 sobre a aplicação da álgebra booleana aos circuitos de comutação.

A completude funcional, portanto, não é uma questão meramente formal: é ela que garante, de um lado, que as formas normais definidas na Seção 1.3 são capazes de representar qualquer fórmula da lógica proposicional, e, de outro, que a escolha da base de conectivos empregada em uma implementação computacional — seja ela $\{\wedge, \vee, \neg\}$, seja um conectivo único como o NAND — é uma decisão de engenharia e não uma limitação expressiva. Essa distinção entre expressividade teórica e economia de representação será retomada na Seção 1.5, na qual são discutidas as limitações teóricas que, apesar da completude funcional aqui estabelecida, ainda cercam o uso prático das formas normais.

A Figura 1.1 situa, numa mesma linha do tempo, os marcos históricos discutidos ao longo das Seções 1.1, 1.2 e 1.4 — desde a álgebra de Boole até os resultados de Cook sobre a complexidade do problema de satisfazibilidade, retomado na Seção 1.5.

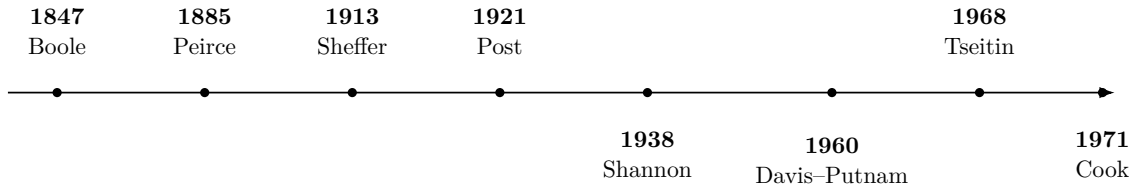


Figura 1.1: Linha do tempo (fora de escala) dos principais marcos históricos discutidos no Capítulo 1

1.5 Limitações teóricas

As três seções anteriores estabeleceram um panorama aparentemente favorável: toda fórmula proposicional admite uma NNF, uma CNF e uma DNF equivalentes (Seção 1.3), toda fórmula de primeira ordem admite uma PNF equivalente (Seção 1.3), e a completude

funcional garante que qualquer base de conectivos suficientemente expressiva — desde $\{\wedge, \vee, \neg\}$ até um único conectivo como o NAND — é capaz de sustentar essa representação (Seção 1.4). Esse panorama, no entanto, diz respeito apenas à existência das formas normais, não ao custo de sua obtenção nem à sua utilidade prática irrestrita. Cabe agora fechar o Capítulo 1 examinando as limitações teóricas que atravessam esse aparato, as quais motivam, em grande medida, os algoritmos de conversão que serão discutidos no Capítulo 2.

A primeira e mais imediata dessas limitações é de natureza combinatória: a conversão ingênua de uma fórmula qualquer para a forma normal conjuntiva ou disjuntiva, por meio da aplicação repetida das leis distributivas sobre uma fórmula em NNF, pode acarretar um crescimento exponencial no número de cláusulas ou de termos em relação ao tamanho da fórmula original. Uma fórmula tão simples quanto $(a_1 \wedge b_1) \vee (a_2 \wedge b_2) \vee \cdots \vee (a_n \wedge b_n)$, já em DNF, produz, quando convertida para CNF por distributividade, uma conjunção com 2^n cláusulas. Esse problema levou Grigori Tseitin a propor, em 1968, um procedimento alternativo de conversão para CNF que introduz uma variável auxiliar para cada subfórmula da expressão original, produzindo uma fórmula cujo tamanho cresce apenas linearmente em relação ao da fórmula de entrada [15]. O preço dessa eficiência, contudo, é uma perda de equivalência estrita: a fórmula obtida pela transformação de Tseitin não é logicamente equivalente à fórmula original, mas apenas *equissatisfazível* a ela — isto é, uma é satisfazível se e somente se a outra também o for —, o que basta para os propósitos de verificação de satisfazibilidade, mas não para todo e qualquer uso das formas normais.

A Figura 1.2 resume as rotas de conversão discutidas até aqui, contrastando o caminho tradicional via distributividade — que preserva a equivalência lógica, mas pode crescer exponencialmente — com a alternativa de Tseitin, que preserva apenas a equissatisfazibilidade, mas em tamanho linear; a figura inclui ainda o desvio próprio da PNF para fórmulas de primeira ordem, retomado ao final desta seção.

Além do custo de obtenção, há um limite teórico mais profundo relacionado à complexidade computacional dos problemas que essas formas normais foram concebidas para resolver. Stephen Cook demonstrou, em 1971, que o problema de decidir se uma fórmula em CNF é satisfazível — o problema SAT mencionado na Seção 1.2 — é NP-completo, isto é, pertence à classe de problemas mais difíceis dentre aqueles cuja solução pode ser verificada em tempo polinomial [4]. Esse resultado, seminal para a teoria da complexidade computacional, tem uma contrapartida dual e frequentemente menos discutida: decidir se uma fórmula em DNF é uma tautologia — ou seja, se é verdadeira sob toda atribuição de valores — é um problema coNP-completo. A CNF e a DNF, portanto, embora simétricas do ponto de vista sintático, são radicalmente assimétricas do ponto de vista computacional: cada uma delas é eficiente para decidir um tipo de pergunta (satisfazibilidade, no caso da CNF; validade, no caso da DNF) e, presumivelmente, ineficiente para decidir a

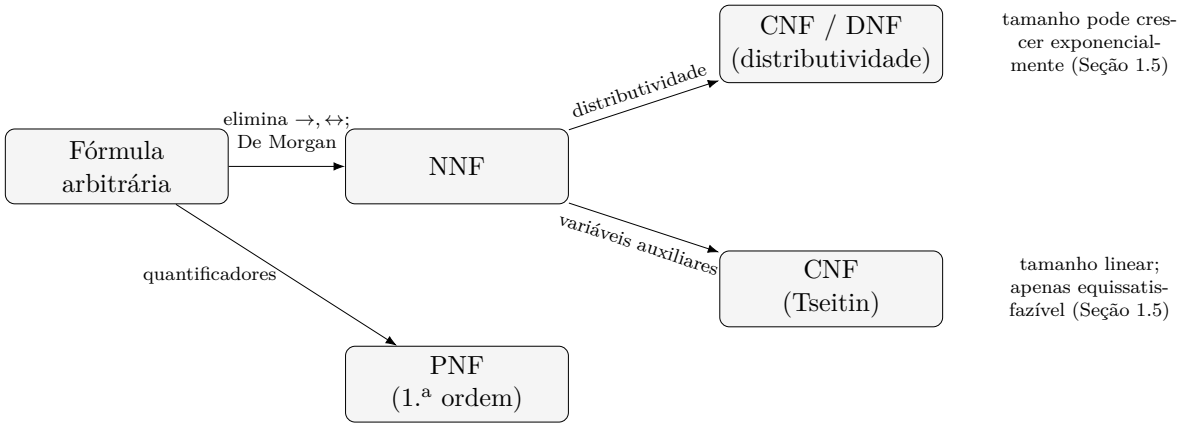


Figura 1.2: Rotas de conversão de uma fórmula arbitrária para formas normais, com a alternativa de Tseitin

pergunta dual, a menos que P = NP.

Uma terceira limitação diz respeito à ausência de um procedimento eficiente para obter a forma normal *mínima* de uma fórmula, isto é, a CNF ou a DNF com o menor número possível de cláusulas ou de termos. Embora a existência de uma forma normal esteja garantida pelos teoremas discutidos na Seção 1.3, encontrar a representação mais compacta entre as múltiplas formas normais equivalentes de uma mesma fórmula — problema historicamente associado à minimização de circuitos lógicos e a métodos como o de Quine–McCluskey — é, em geral, um problema computacionalmente intratável. A não unicidade das formas normais, já apontada na Seção 1.3, não é, portanto, apenas uma curiosidade formal, mas a raiz de um problema de otimização genuinamente difícil.

Por fim, no domínio da lógica de primeira ordem, a forma normal prenexa apresenta uma limitação análoga à observada na transformação de Tseitin: o procedimento de skolemização, frequentemente empregado em conjunto com a PNF para eliminar quantificadores existenciais em provadores automáticos de teoremas, também não preserva a equivalência lógica da fórmula original, apenas sua equissatisfazibilidade, e pode introduzir símbolos funcionais que aumentam a assinatura da linguagem. Além disso, a ordem em que os quantificadores aparecem no prefixo de uma PNF não é arbitrária: alternâncias entre quantificadores universais e existenciais podem impor, a certas classes de problemas, limites inferiores de complexidade que nenhuma reformulação sintática é capaz de contornar.

Em conjunto, essas limitações — o crescimento exponencial da conversão ingênua, a intratabilidade computacional subjacente aos problemas de satisfazibilidade e validade, a dificuldade de minimização e a perda de equivalência estrita nos procedimentos de skolemização — não invalidam a utilidade teórica e prática das formas normais estabelecida ao longo deste capítulo, mas delimitam com precisão o que se pode esperar delas. É

justamente esse delicado equilíbrio entre elegância teórica e custo computacional que o Capítulo 2 se propõe a examinar em maior profundidade, ao revisar os principais algoritmos de conversão para formas normais e discutir sua correção, sua complexidade e as decisões de implementação que deles decorrem.

Capítulo 2

Algoritmos de Transformação

2.1 Revisão dos principais métodos de conversão para formas normais

O Capítulo 1 estabeleceu a existência das formas normais e apontou, em termos gerais, os custos e as limitações que cercam sua obtenção. Cabe agora revisar, de maneira mais detida, os principais métodos algorítmicos empregados na conversão de uma fórmula arbitrária para cada uma dessas formas, preparando o terreno para a análise de correção e de complexidade computacional que será conduzida na Seção 2.2.

O primeiro estágio de qualquer um desses métodos é, invariavelmente, a obtenção da forma normal negativa. O algoritmo clássico de conversão para NNF procede em duas etapas recursivas sobre a estrutura da fórmula: a eliminação dos conectivos de implicação e de bicondicional, reescritos em termos de $\wedge$, $\vee$ e $\neg$, seguida da aplicação das leis de De Morgan e da eliminação da dupla negação para empurrar as negações até os literais. Para fórmulas construídas com $\neg$, $\wedge$, $\vee$ e $\rightarrow$, trata-se de um procedimento estrutural, aplicado uma única vez a cada subfórmula, cujo custo é linear no tamanho da fórmula de entrada — uma propriedade que o distingue favoravelmente dos métodos discutidos a seguir e que justifica sua adoção como etapa preparatória universal. A presença do bicondicional exige cautela, pois sua reescrita em termos de $\wedge$ e $\vee$ duplica ambos os operandos, o que pode produzir crescimento exponencial em fórmulas com bicondicionais aninhados; esse ponto é examinado na Seção 2.2.

A partir de uma fórmula em NNF, o método tradicional de obtenção da CNF e da DNF consiste na aplicação repetida das leis distributivas, tal como apresentado na Seção 1.3. Esse método, por vezes chamado de conversão estrutural ingênua, tem a vantagem de preservar a equivalência lógica estrita da fórmula original, mas, como discutido na Seção 1.5, pode acarretar um crescimento exponencial no número de cláusulas ou de termos, o que o torna impraticável para fórmulas de porte considerável.

Como alternativa, Tseitin propôs, em 1968, um método de conversão que introduz uma variável auxiliar para cada subfórmula não literal da expressão original, associando a cada uma delas uma bi-implicação que define seu valor em função dos valores de suas subfórmulas imediatas [15]. A conjunção de todas essas definições, já naturalmente em CNF, produz uma fórmula de tamanho linear em relação à fórmula de entrada — ao custo, já discutido na Seção 1.5, de preservar apenas a equissatisfazibilidade, e não a equivalência lógica estrita, com a fórmula original. Uma extensão importante desse método foi proposta por Plaisted e Greenbaum em 1986, que observaram ser possível reduzir ainda mais o número de cláusulas geradas ao levar em conta a *polaridade* de cada subfórmula — isto é, a posição em que ela ocorre relativamente aos operadores de negação que a envolvem —, permitindo que, para subfórmulas de polaridade única, apenas uma das duas direções da bi-implicação seja de fato necessária [10]. Essa otimização, hoje amplamente empregada em provadores de teoremas e em solucionadores de SAT, ilustra como as duas décadas que separam Tseitin de Plaisted e Greenbaum foram dedicadas não a superar a compensação fundamental entre equivalência estrita e tamanho linear, mas a extrair dela o máximo de economia possível.

No que diz respeito à forma normal prenexa, o método de conversão consiste na aplicação sucessiva de regras de equivalência que autorizam mover um quantificador para fora do escopo de um conectivo proposicional — por exemplo, transformando $\neg\forall x\, P(x)$ em $\exists x\, \neg P(x)$, ou $P \wedge \forall x\, Q(x)$, com x não ocorrendo livre em P, em $\forall x\, (P \wedge Q(x))$ —, sempre precedidas da renomeação de variáveis ligadas que compartilhem o mesmo nome, de modo a evitar capturas indevidas. Esse procedimento, aplicado exaustivamente, sempre termina em uma fórmula em PNF, mas a ordem em que os quantificadores são movidos não é única, e diferentes estratégias de aplicação das regras podem produzir prefixos de quantificadores distintos — uma observação que retoma a discussão sobre não unicidade das formas normais apresentada na Seção 1.3, e que ganha relevância especial no caso da PNF, uma vez que a ordem dos quantificadores pode afetar diretamente a eficiência dos procedimentos de prova automática construídos sobre ela.

Por fim, cabe mencionar os métodos voltados não à obtenção de uma forma normal qualquer, mas à obtenção de sua versão *mínima*. O mais conhecido deles é o método de Quine–McCluskey, proposto por Willard Quine em 1952 [12] e sistematizado por Edward McCluskey em 1956 [7], que procede por tabulação: a partir dos mintermos de uma função booleana, o método agrupa e combina sucessivamente os termos que diferem em exatamente um literal, produzindo o conjunto de implicantes primos da função, do qual se extrai, por meio de uma carta de cobertura, a expressão mínima em soma de produtos (ou, dualmente, em produto de somas). Trata-se de um método determinístico e sistemático, superior em rigor ao método gráfico dos mapas de Karnaugh para funções com muitas variáveis, mas que, como já apontado na Seção 1.5, tem custo exponencial no pior caso

— uma limitação inerente ao próprio problema de minimização, e não a uma deficiência específica do método.

Os métodos revisados nesta seção — conversão para NNF, distributividade, Tseitin, Plaisted–Greenbaum, prenexação e Quine–McCluskey — serão retomados na Seção 2.2, na qual sua correção será formalmente demonstrada e sua complexidade computacional analisada com maior rigor.

2.2 Correção, equivalência lógica e complexidade computacional

A revisão conduzida na Seção 2.1 descreveu o que cada método faz; cabe agora estabelecer o que cada um deles garante e quanto custa garantir. Três propriedades orientam essa análise: a *terminação*, isto é, a certeza de que o procedimento alcança uma fórmula na forma-alvo após um número finito de passos; a *correção*, entendida como a relação semântica que a fórmula produzida mantém com a de entrada; e a *complexidade*, medida tanto no tempo de execução quanto no tamanho da saída. No que diz respeito à correção, convém distinguir duas noções que a Seção 1.5 já havia contraposto: a **equivalência lógica**, segundo a qual duas fórmulas são verdadeiras sob exatamente as mesmas valorações, e a **equissatisfazibilidade**, segundo a qual uma é satisfazível se e somente se a outra o é. A primeira implica a segunda, mas não o contrário, e a diferença entre ambas é precisamente o que permite a certos métodos escapar do crescimento exponencial, como se verá adiante.

Comecemos pela forma normal negativa, cujo procedimento convém formular de maneira recursiva, carregando-se consigo a *polaridade* $\pi \in \{+, -\}$ da subfórmula corrente, isto é, se ela deve ser lida afirmada ($+$) ou negada ($-$). Escrevendo $\overline{\pi}$ para a polaridade oposta a π e p para uma variável proposicional, a função nnf é definida por

$$
\begin{aligned}
\operatorname{nnf}(p, +) &= p, \qquad \operatorname{nnf}(p, -) = \neg p, \\
\operatorname{nnf}(\neg\varphi, \pi) &= \operatorname{nnf}(\varphi, \overline{\pi}), \\
\operatorname{nnf}(\varphi \wedge \psi, +) &= \operatorname{nnf}(\varphi, +) \wedge \operatorname{nnf}(\psi, +), \\
\operatorname{nnf}(\varphi \wedge \psi, -) &= \operatorname{nnf}(\varphi, -) \vee \operatorname{nnf}(\psi, -), \\
\operatorname{nnf}(\varphi \vee \psi, +) &= \operatorname{nnf}(\varphi, +) \vee \operatorname{nnf}(\psi, +), \\
\operatorname{nnf}(\varphi \vee \psi, -) &= \operatorname{nnf}(\varphi, -) \wedge \operatorname{nnf}(\psi, -), \\
\operatorname{nnf}(\varphi \to \psi, +) &= \operatorname{nnf}(\varphi, -) \vee \operatorname{nnf}(\psi, +), \\
\operatorname{nnf}(\varphi \to \psi, -) &= \operatorname{nnf}(\varphi, +) \wedge \operatorname{nnf}(\psi, -), \\
\operatorname{nnf}(\varphi \leftrightarrow \psi, +) &= \Big(\operatorname{nnf}(\varphi, -) \vee \operatorname{nnf}(\psi, +)\Big) \wedge \Big(\operatorname{nnf}(\psi, -) \vee \operatorname{nnf}(\varphi, +)\Big),
\end{aligned}
$$

$$\mathrm{nnf}(\varphi \leftrightarrow \psi, -) = \Big(\mathrm{nnf}(\varphi, +) \wedge \mathrm{nnf}(\psi, -)\Big) \vee \Big(\mathrm{nnf}(\psi, +) \wedge \mathrm{nnf}(\varphi, -)\Big).$$

Essa formulação funde, em uma única passagem, as duas etapas descritas na Seção 2.1: a eliminação dos conectivos $\rightarrow$ e $\leftrightarrow$ e o deslocamento das negações até os literais.

Proposição 2.1 (Correção da NNF)**.** *Para toda fórmula φ,* $\mathrm{nnf}(\varphi, +)$ *está em NNF e é logicamente equivalente a φ, e* $\mathrm{nnf}(\varphi, -)$ *está em NNF e é logicamente equivalente a $\neg\varphi$. Além disso, o cálculo termina e, se φ não contém $\leftrightarrow$, seu custo, em tempo e em tamanho da saída, é linear em $|\varphi|$.*

Demonstração. A prova é por indução estrutural em φ, simultaneamente para as duas polaridades. Que o resultado está em NNF decorre de a negação ser introduzida apenas diante de variáveis, e a terminação decorre de cada chamada recursiva incidir sobre uma subfórmula própria. Quanto à equivalência, o caso de variável é imediato. No caso $\neg\varphi$, a hipótese de indução dá $\mathrm{nnf}(\neg\varphi, +) = \mathrm{nnf}(\varphi, -) \equiv \neg\varphi$ e $\mathrm{nnf}(\neg\varphi, -) = \mathrm{nnf}(\varphi, +) \equiv \varphi \equiv \neg\neg\varphi$. Para a conjunção com polaridade negativa, $\mathrm{nnf}(\varphi, -) \vee \mathrm{nnf}(\psi, -) \equiv \neg\varphi \vee \neg\psi \equiv \neg(\varphi \wedge \psi)$ pela lei de De Morgan; para a implicação com polaridade positiva, $\mathrm{nnf}(\varphi, -) \vee \mathrm{nnf}(\psi, +) \equiv \neg\varphi \vee \psi \equiv \varphi \rightarrow \psi$, e com polaridade negativa, $\mathrm{nnf}(\varphi, +) \wedge \mathrm{nnf}(\psi, -) \equiv \varphi \wedge \neg\psi \equiv \neg(\varphi \rightarrow \psi)$. Os demais casos são análogos. Por fim, na ausência de $\leftrightarrow$, cada subfórmula de φ recebe exatamente uma chamada, de modo que o número de chamadas, e portanto o tamanho da saída, é linear em $|\varphi|$. □

A ressalva quanto ao bicondicional não é meramente técnica. Na regra de $\leftrightarrow$, cada operando é percorrido, e copiado, duas vezes, uma em cada polaridade; em uma fórmula com bicondicionais aninhados em profundidade k, o tamanho da saída ao menos dobra a cada nível de aninhamento, atingindo a ordem de 2^k. A correção do método estrutural é, portanto, incondicional, ao passo que sua eficiência não o é. Essa duplicação de operandos reaparecerá adiante como motivação para o tratamento direto de equivalências nos métodos de Tseitin e de Plaisted–Greenbaum.

Passemos à obtenção da CNF a partir da NNF por distributividade. Sua correção repousa sobre a equivalência $(\alpha \wedge \beta) \vee \gamma \equiv (\alpha \vee \gamma) \wedge (\beta \vee \gamma)$ e sobre a substituição de subfórmulas por equivalentes. Na formulação recursiva, $\mathrm{cnf}(\alpha \wedge \beta) = \mathrm{cnf}(\alpha) \wedge \mathrm{cnf}(\beta)$ e $\mathrm{cnf}(\alpha \vee \beta) = \mathrm{cnf}(\alpha) \otimes \mathrm{cnf}(\beta)$, em que $\otimes$ denota o conjunto das disjunções de uma cláusula do primeiro argumento com uma cláusula do segundo; a terminação é imediata e a equivalência segue por indução estrutural. É o produto $\otimes$ que explica o custo: se $\mathrm{cnf}(\alpha)$ tem m cláusulas e $\mathrm{cnf}(\beta)$ tem k, o resultado tem $m \cdot k$. Aplicado à fórmula $\varphi_n = (a_1 \wedge b_1) \vee \cdots \vee (a_n \wedge b_n)$, de tamanho $O(n)$, cada termo $a_i \wedge b_i$ contribui com duas cláusulas unitárias, e os $n-1$ produtos sucessivos resultam em 2^n cláusulas. A proposição a seguir mostra que esse crescimento não é um artefato do método.

Proposição 2.2 (Limite inferior para a CNF)**.** *Para todo $n \geq 1$, toda CNF equivalente a $\varphi_n = (a_1 \wedge b_1) \vee \cdots \vee (a_n \wedge b_n)$ possui ao menos 2^n cláusulas.*

Demonstração. Seja Γ uma CNF equivalente a φ_n, da qual se descartaram previamente as cláusulas tautológicas. A fórmula φ_n é monótona: se uma valoração a satisfaz, qualquer valoração que torne verdadeiras ao menos as mesmas variáveis também a satisfaz. Consequentemente, se uma cláusula $C \vee \neg x$, com x não ocorrendo em C, é implicada por φ_n, então C também o é: dada $v \models \varphi_n$, a valoração v', que difere de v apenas por tornar x verdadeira, também satisfaz φ_n, logo satisfaz $C \vee \neg x$ e, como $\neg x$ é falso em v', satisfaz C; como x não ocorre em C, v também satisfaz C. Removendo, um a um, os literais negativos das cláusulas de Γ, obtém-se uma CNF positiva Γ', com no máximo $|\Gamma|$ cláusulas, equivalente a φ_n: cada cláusula de Γ' é implicada por φ_n, e Γ' implica Γ, pois cada uma de suas cláusulas é subcláusula de uma cláusula de Γ.

Para cada $s \in \{0,1\}^n$, seja C_s a cláusula positiva que contém, para cada i, a variável a_i se $s_i = 0$ e b_i se $s_i = 1$, e seja v_s a valoração que atribui falso às variáveis de C_s e verdadeiro às demais. Em v_s, exatamente uma entre a_i e b_i é verdadeira, para cada i; logo nenhum termo $a_i \wedge b_i$ é verdadeiro e $v_s \not\models \varphi_n$. Assim, alguma cláusula D de Γ' é falsa em v_s e, sendo positiva, todas as suas variáveis pertencem a C_s. Por outro lado, D é implicada por φ_n, e a valoração que torna verdadeiras apenas a_i e b_i satisfaz φ_n; logo D contém a_i ou b_i, para cada i. Conclui-se que $D = C_s$. Como os 2^n valores de s produzem cláusulas C_s distintas, $|\Gamma| \geq |\Gamma'| \geq 2^n$. □

Por dualidade, a fórmula $(a_1 \vee b_1) \wedge \cdots \wedge (a_n \vee b_n)$ exige ao menos 2^n termos em qualquer DNF equivalente. O resultado tem uma consequência direta: nenhum algoritmo que preserve a equivalência lógica e produza uma CNF (ou uma DNF) pode ter tamanho de saída polinomial no caso geral. O custo exponencial da distributividade é, assim, uma propriedade da exigência de equivalência, e não uma deficiência do método.

É exatamente essa exigência que o método de Tseitin abandona. Para fixar a notação, consideremos φ em NNF, construída com $\wedge$ e $\vee$ binários sobre literais. A cada ocorrência ψ de subfórmula não literal associa-se uma variável nova x_ψ, e escreve-se $[\psi]$ para o literal que representa ψ: $[\psi] = \psi$ se ψ é um literal, e $[\psi] = x_\psi$ caso contrário. Denotando por $\overline{\ell}$ o literal complementar de ℓ, o conjunto de cláusulas associado a uma ocorrência $\psi = \psi_1 \wedge \psi_2$ é

$$G_\psi = \Big\{ \neg x_\psi \vee [\psi_1],\ \ \neg x_\psi \vee [\psi_2],\ \ x_\psi \vee \overline{[\psi_1]} \vee \overline{[\psi_2]} \Big\},$$

e o de uma ocorrência $\psi = \psi_1 \vee \psi_2$ é

$$G_\psi = \Big\{ \neg x_\psi \vee [\psi_1] \vee [\psi_2],\ \ x_\psi \vee \overline{[\psi_1]},\ \ x_\psi \vee \overline{[\psi_2]} \Big\}.$$

Em ambos os casos, G_ψ é a CNF da bi-implicação que define x_ψ em função de seus

operandos, isto é, de $x_\psi \leftrightarrow ([\psi_1] \wedge [\psi_2])$ e de $x_\psi \leftrightarrow ([\psi_1] \vee [\psi_2])$, respectivamente. A transformação é $T(\varphi) = x_\varphi \wedge \bigwedge_\psi G_\psi$, com a convenção de que $T(\varphi) = \varphi$ quando φ é um literal [15].

Proposição 2.3 (Correção da transformação de Tseitin)**.** *Seja φ uma fórmula em NNF, com conectivos binários, sobre o conjunto de variáveis V, e seja X o conjunto das variáveis auxiliares de $T(\varphi)$. Então: (i) $T(\varphi)$ está em CNF e possui no máximo $3c + 1$ cláusulas, em que c é o número de ocorrências de conectivos em φ, de modo que seu tamanho e seu tempo de construção são lineares em $|\varphi|$; (ii) toda valoração de V que satisfaz φ admite uma única extensão a $V \cup X$ que satisfaz $T(\varphi)$; (iii) toda valoração de $V \cup X$ que satisfaz $T(\varphi)$ satisfaz φ. Consequentemente, φ e $T(\varphi)$ são equissatisfazíveis e têm o mesmo número de modelos (contados sobre V e sobre $V \cup X$, respectivamente); além disso, $T(\varphi) \models \varphi$, mas em geral $\varphi \not\models T(\varphi)$.*

Demonstração. (i) Cada ocorrência de conectivo gera três cláusulas com no máximo três literais cada, às quais se acrescenta a cláusula unitária x_φ. Para (ii) e (iii), a peça central é o seguinte lema: se J satisfaz todas as cláusulas G_ψ, então $J(x_\psi) = J(\psi)$ para toda ocorrência não literal ψ. A prova é por indução estrutural em ψ. Se $\psi = \psi_1 \wedge \psi_2$, a satisfação de G_ψ equivale a $J(x_\psi) = J([\psi_1]) \wedge J([\psi_2])$, e $J([\psi_i]) = J(\psi_i)$, pela hipótese de indução quando ψ_i não é literal e trivialmente quando o é; logo $J(x_\psi) = J(\psi)$. O caso da disjunção é análogo.

(ii) Dada $I \models \varphi$, defina-se I' por $I'(x_\psi) = I(\psi)$. Cada bi-implicação vale por construção e $I'(x_\varphi) = I(\varphi) = 1$, de modo que $I' \models T(\varphi)$. A unicidade segue do lema, que determina o valor de cada x_ψ.

(iii) Se J satisfaz $T(\varphi)$, então $J(x_\varphi) = 1$ e, pelo lema, $J(\varphi) = J(x_\varphi) = 1$.

Por fim, (ii) e (iii) fornecem a equissatisfazibilidade e, por estabelecerem uma correspondência biunívoca entre os modelos, a igualdade de suas quantidades; (iii) é a implicação $T(\varphi) \models \varphi$. Para a recíproca, tome-se $\varphi = p \wedge q$: então $T(\varphi) = x \wedge (\neg x \vee p) \wedge (\neg x \vee q) \wedge (x \vee \neg p \vee \neg q)$, e a valoração $p = q = 1$, $x = 0$ satisfaz φ, mas não $T(\varphi)$. □

A construção estende-se a fórmulas arbitrárias tratando cada ocorrência de conectivo como uma porta com um número constante de cláusulas, inclusive para o bicondicional, que requer quatro. Esse tratamento direto evita a duplicação de operandos que torna a conversão para NNF exponencial em presença de bicondicionais aninhados. O refinamento de Plaisted e Greenbaum [10] explora, por sua vez, o fato de que, em uma fórmula em NNF, toda ocorrência não literal tem polaridade positiva: basta, então, codificar a direção $x_\psi \rightarrow ([\psi_1] \wedge [\psi_2])$, ou $x_\psi \rightarrow ([\psi_1] \vee [\psi_2])$, de modo que cada porta gera duas cláusulas, no caso da conjunção, ou uma, no da disjunção, em vez de três. A prova de correção segue a da Proposição 2.3 com o lema enfraquecido para $J(x_\psi) = 1 \Rightarrow J(\psi) = 1$, o que basta para

(iii); a extensão de (ii) continua a existir, mas deixa de ser única. A equissatisfazibilidade é, assim, preservada, ao passo que o número de modelos, em geral, não o é.

No caso da forma normal prenexa, a correção apoia-se em duas famílias de equivalências: as regras de deslocamento de quantificadores, que só são equivalências sob a condição lateral de que a variável quantificada não ocorra livre na outra componente — por exemplo, $\psi \wedge \forall x\, \varphi \equiv \forall x\, (\psi \wedge \varphi)$ se $x \notin \mathrm{FV}(\psi)$, em que $\mathrm{FV}(\psi)$ denota o conjunto das variáveis que ocorrem livres em ψ —, e a renomeação de variáveis ligadas (α-conversão), que, aplicada previamente, permite satisfazer sempre essa condição. Como cada aplicação desloca um quantificador exatamente uma posição para fora e nenhuma regra o desloca para dentro, o procedimento termina após no máximo $q \cdot d$ deslocamentos, em que q é o número de quantificadores e d a profundidade da fórmula. O custo é, assim, polinomial, e o tamanho da fórmula não cresce, com a mesma ressalva, relativa ao bicondicional, que se fez para a NNF. A skolemização, que frequentemente se segue à prenexação, preserva apenas a equissatisfazibilidade (Seção 1.5), situando-se, nesse aspecto, do mesmo lado da fronteira que a transformação de Tseitin.

Resta o método de Quine–McCluskey. Sua correção decorre do resultado de Quine segundo o qual alguma soma de produtos mínima é formada apenas por implicantes primos [12, 7]. Quanto ao custo, convém distinguir as duas fases do método. A geração dos implicantes primos produz, para n variáveis, uma quantidade de implicantes limitada por uma função da ordem de $3^n/\sqrt{n}$, limite quase justo, pois existem funções com implicantes primos da ordem de $3^n/n$ [2]. Como a tabela-verdade tem $N = 2^n$ linhas e $3^n = N^{\log_2 3}$, esse número é polinomial em N, embora exponencial em n. A dificuldade essencial reside na segunda fase, a seleção, entre os implicantes primos, de uma cobertura de tamanho mínimo, problema do tipo cobertura de conjuntos: minimizar uma DNF dada a tabela-verdade é NP-completo, resultado atribuído a Masek, nunca publicado, e reapresentado com uma demonstração mais simples por Allender et al. [1].

Em síntese, os resultados desta seção delimitam, para cada método, o que se obtém e a que preço. A conversão para NNF e para PNF preserva a equivalência a custo polinomial, linear quando não há bicondicionais; a distributividade preserva a equivalência, mas está sujeita ao limite inferior exponencial da Proposição 2.2; as transformações de Tseitin e de Plaisted–Greenbaum atingem tamanho linear ao custo de trocar a equivalência pela equissatisfazibilidade; e a obtenção de formas mínimas esbarra na NP-completude da seleção de cobertura. Esse quadro de compromissos condiciona o que uma implementação pode oferecer a um estudante, em particular se a ferramenta deve exibir apenas transformações que preservam a equivalência, que se acompanham passo a passo com mais facilidade, ou também transformações equissatisfazíveis, mais eficientes, porém menos transparentes do ponto de vista didático. Essa escolha de projeto será retomada na Seção 2.3 e no Capítulo 3.

2.3 Considerações sobre implementação computacional

Os métodos examinados nas seções anteriores foram tratados como objetos matemáticos: procedimentos definidos sobre fórmulas, cuja correção e cujo custo foram estabelecidos em abstrato. Cabe agora considerar o que se altera quando esses procedimentos passam a ser programas, e em particular programas destinados a acompanhar um estudante em cada etapa de uma transformação. A implementação introduz decisões que a análise teórica deixa em aberto, como a representação das fórmulas, a ordem de aplicação das regras e o tratamento dos casos em que o resultado é grande demais para ser exibido, e algumas dessas decisões decorrem diretamente dos resultados da Seção 2.2.

A primeira delas é a delimitação do que a ferramenta se propõe a fazer. Dos métodos revisados na Seção 2.1, a ferramenta restringe-se à conversão para a forma normal conjuntiva e para a forma normal disjuntiva. A forma normal negativa figura como etapa intermediária indispensável, uma vez que ambas as conversões partem dela, mas não como produto final autônomo; a forma normal prenexa, a skolemização e a minimização por Quine–McCluskey permanecem como objeto de análise teórica neste trabalho, sem módulo correspondente. O recorte concentra a implementação no domínio proposicional, no qual cada passo da conversão é uma aplicação de uma lei algébrica elementar e pode, por isso, ser exibido e conferido por quem estuda.

No que diz respeito à representação, o caminho natural é tratar cada fórmula como uma árvore de sintaxe abstrata, com nós para variáveis, negação e conectivos binários. Se essas árvores são imutáveis, cada passo da conversão produz uma nova fórmula sem alterar a anterior, e a derivação completa, isto é, a sequência de fórmulas intermediárias acompanhadas da lei aplicada em cada passo, pode ser conservada integralmente, o que é precisamente o que a interface pedagógica precisa exibir. A entrada textual exige, além disso, um analisador sintático que fixe a precedência e a associatividade dos conectivos; a convenção usual, em que a negação se liga mais fortemente que a conjunção, esta mais que a disjunção, e esta mais que a implicação e o bicondicional, é um dos pontos a serem fixados e documentados na Seção 3.1, pois uma convenção implícita é fonte frequente de leituras divergentes entre o estudante e a ferramenta.

A representação do resultado merece atenção própria. Uma CNF pode ser vista como uma árvore, fiel à notação lógica, ou como um conjunto de conjuntos de literais, a chamada forma clausal, na qual a ordem e a repetição de cláusulas e de literais deixam de existir. A segunda representação é mais compacta e torna triviais as comparações, mas incorpora silenciosamente as leis da comutatividade, da associatividade e da idempotência, que a árvore trata como transformações explícitas. Essa diferença não é inócua em um contexto de ensino: se a ferramenta usa a forma clausal durante toda a derivação, certos passos que

o estudante faria à mão desaparecem sem registro. Há, portanto, um compromisso entre a economia da representação e a transparência da derivação, que é resolvido, no projeto, mantendo-se a estrutura de árvore enquanto a derivação é exibida. A forma clausal permanece útil para as simplificações opcionais que preservam a equivalência, como a eliminação de cláusulas tautológicas e de cláusulas subsumidas por outras (e, dualmente, de termos contraditórios e absorvidos, no caso da DNF).

Do ponto de vista do procedimento, a conversão pode ser vista como um sistema de reescrita, em que cada passo consiste em aplicar uma lei, como a eliminação de implicações, as leis de De Morgan, a eliminação da dupla negação ou a distributividade, a uma ocorrência determinada de subfórmula. Como a forma normal de uma fórmula não é única (Seção 1.3), a ordem em que as ocorrências são escolhidas influencia o resultado e o tamanho das fórmulas intermediárias. Uma implementação voltada ao ensino precisa, por isso, adotar uma estratégia determinística, que sempre escolha a ocorrência segundo o mesmo critério, de modo que a mesma entrada produza sempre a mesma derivação, o que é requisito tanto para a reprodutibilidade em sala de aula quanto para a escrita de testes automatizados. O pseudocódigo dos procedimentos de conversão para a CNF e para a DNF, incluindo o tratamento do limite de tamanho, consta do Apêndice A.

A Proposição 2.2 tem uma consequência prática que nenhuma implementação pode contornar: como toda CNF equivalente a $(a_1 \wedge b_1) \vee \cdots \vee (a_n \wedge b_n)$ tem ao menos 2^n cláusulas, já para $n = 20$ a saída ultrapassa um milhão de cláusulas, e o mesmo vale, dualmente, para a DNF. Uma ferramenta que preserva a equivalência encontrará, portanto, entradas para as quais o resultado é grande demais para ser exibido ou, simplesmente, para ser útil a quem estuda. A resposta adequada não é deixar que a execução se esgote em memória, mas impor um limite ao tamanho da fórmula e ao número de passos, e informar o estudante de que o limite foi atingido, de modo que a explosão combinatória seja apresentada como o fenômeno que é, e não como uma falha do programa. O caso do bicondicional aninhado na NNF admite um tratamento mais fino: como a função nnf da Seção 2.2 depende apenas da subfórmula e da polaridade, e cada subfórmula admite somente duas polaridades, o armazenamento de resultados já calculados permite que as cópias sejam compartilhadas em memória, de modo que a representação interna cresce linearmente, embora a fórmula exibida continue exponencial.

A verificação do resultado é um problema à parte, e sua solução fundamenta a validação descrita na Seção 3.4. Há três propriedades a conferir: que a saída esteja de fato na forma-alvo, o que é um teste puramente sintático; que ela seja equivalente à entrada; e que cada passo intermediário aplique corretamente a lei declarada. A equivalência pode ser verificada, para fórmulas com poucas variáveis, comparando-se as tabelas-verdade, ao custo de 2^n valorações; para fórmulas maiores, recorre-se à redução ao problema SAT, uma vez que $\varphi \equiv \psi$ se e somente se $\neg(\varphi \leftrightarrow \psi)$ é insatisfazível. A tabela-verdade tem a

vantagem adicional de ser, ela própria, um método de conversão (Seção 1.3) e, portanto, um oráculo independente do algoritmo implementado.

As transformações de Tseitin e de Plaisted–Greenbaum levantam uma questão distinta. Ambas produzem uma CNF de tamanho linear, mas ao preço de introduzir variáveis auxiliares e de preservar apenas a equissatisfazibilidade, de modo que a verificação de equivalência descrita acima deixa de se aplicar a seus resultados e a derivação se afasta da fórmula original de maneira que um estudante não reconstitui por manipulação algébrica. Se a ferramenta deve oferecê-las, como alternativa à distributividade nos casos em que esta é inviável, é matéria a ser definida junto aos requisitos funcionais, na Seção 3.1.

A Tabela 2.1 reúne os resultados da Seção 2.2 segundo os critérios que orientam as decisões acima: a forma obtida, a relação que ela mantém com a fórmula original e o tamanho e o custo envolvidos. A leitura conjunta das colunas evidencia o compromisso central do capítulo: os métodos que preservam a equivalência, e que por isso se prestam a uma derivação algébrica passo a passo, são justamente aqueles sujeitos a crescimento exponencial no caso da CNF e da DNF, ao passo que os de custo linear abandonam a equivalência. É sobre esse quadro que se constroem os requisitos e a arquitetura da ferramenta, apresentados no Capítulo 3.

Tabela 2.1: Síntese comparativa dos métodos de conversão analisados na Seção 2.2

Método	**Forma obtida**	**Relação com a fórmula original**	**Tamanho da saída e custo**	**Resultado de referência**
Conversão para NNF	NNF	Equivalência	Linear em $\|\varphi\|$ sem $\leftrightarrow$; com bicondicionais aninhados em profundidade k, da ordem de 2^k	Prop. 2.1
Distributividade	CNF ou DNF	Equivalência	Exponencial no pior caso: ao menos 2^n cláusulas para φ_n	Prop. 2.2
Tseitin	CNF	Equissatisfazível	Linear: no máximo $3c + 1$ cláusulas	Prop. 2.3
Plaisted–Greenbaum	CNF	Equissatisfazível	Linear: duas cláusulas por conjunção e uma por disjunção	Seç. 2.2
Prenexação	PNF	Equivalência	Polinomial: no máximo $q \cdot d$ deslocamentos	Seç. 2.2
Quine–McCluskey	DNF ou CNF mínima	Equivalência	Implicantes primos em número da ordem de $3^n/\sqrt{n}$ (polinomial em $N = 2^n$); seleção da cobertura mínima NP-completa	Seç. 2.2

Capítulo 3

Projeto e Implementação da Ferramenta

3.1 Requisitos funcionais e arquitetura da solução

3.2 Módulos de transformação algorítmica

3.3 Interface pedagógica

3.4 Validação: testes automatizados

Capítulo 4

Conclusão

4.1 Síntese dos resultados obtidos

4.2 Contribuições para o ensino de lógica

4.3 Limitações do trabalho realizado

4.4 Perspetivas de evolução e trabalhos futuros

Referências

Bibliografia

[1] Eric Allender, Lisa Hellerstein, Paul McCabe, Toniann Pitassi, and Michael Saks. Minimizing DNF formulas and AC^0_d circuits given a truth table. In *Proceedings of the 21st Annual IEEE Conference on Computational Complexity (CCC 2006)*, 2006. Versão preliminar disponível como ECCC TR05-126 (2005).

[2] Ashok K. Chandra and George Markowsky. On the number of prime implicants. *Discrete Mathematics*, 24(1):7–11, 1978.

[3] Chin-Liang Chang and Richard Char-Tung Lee. *Symbolic Logic and Mechanical Theorem Proving*. Academic Press, New York, 1973.

[4] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC '71)*, pages 151–158, 1971.

[5] Martin Davis and Hilary Putnam. A computing procedure for quantification theory. *Journal of the ACM*, 7(3):201–215, 1960.

[6] Gilson Pedroso dos Santos e Ronilson dos Santos Bezerra. Desenvolvendo o pensamento computacional utilizando scratch e lógica matemática. `https://conteudos.uniasselvi.com.br/extensao/documentos/2023/artigos/7214_9200_1_SM.pdf`, 2017. Acesso em 23 fev. 2026.

[7] Edward J. McCluskey. Minimization of boolean functions. *Bell System Technical Journal*, 35(6):1417–1444, 1956.

[8] Volker Peckhaus. 19th century logic between philosophy and mathematics. *Bulletin of Symbolic Logic*, 5(4):433–450, 1999.

[9] Charles Sanders Peirce. On the algebra of logic: A contribution to the philosophy of notation. *American Journal of Mathematics*, 7(2):180–202, 1885.

[10] David A. Plaisted and Steven Greenbaum. A structure-preserving clause form translation. *Journal of Symbolic Computation*, 2(3):293–304, 1986.

[11] Emil L. Post. Introduction to a general theory of elementary propositions. *American Journal of Mathematics*, 43(3):163–185, 1921.

[12] Willard V. Quine. The problem of simplifying truth functions. *American Mathematical Monthly*, 59(8):521–531, 1952.

[13] Claude E. Shannon. A symbolic analysis of relay and switching circuits. *Transactions of the American Institute of Electrical Engineers*, 57(12):713–723, 1938.

[14] Henry M. Sheffer. A set of five independent postulates for Boolean algebras, with application to logical constants. *Transactions of the American Mathematical Society*, 14(4):481–488, 1913.

[15] Grigori S. Tseitin. On the complexity of derivation in propositional calculus. In *Studies in Constructive Mathematics and Mathematical Logic, Part II*, 1968. Reprinted in J. Siekmann and G. Wrightson (eds.), Automation of Reasoning 2, Springer, 1983, pp. 466–483.

[16] Ketrin Diovana Vargas, João Pablo da Silva, and Alice Finger. Estratégias para o ensino de lógica matemática com pensamento computacional: Uma revisão sistemática da literatura. In *Anais do XXXIII Simpósio Brasileiro de Informática na Educação*, pages 1391–1403, Porto Alegre, RS, Brasil, 2022. SBC.

[17] Michal Walicki. *Introduction to Mathematical Logic*. World Scientific, 2011.

[18] Richard Zach. Logic in mathematics and computer science. 2024. Acesso em 22 mar. 2026.

Apêndice A

Algoritmos de Referência em Pseudocódigo

Este apêndice reúne, em pseudocódigo independente de linguagem, os procedimentos de conversão para as formas normais conjuntiva e disjuntiva, que constituem o escopo da ferramenta descrita no Capítulo 3. Os algoritmos formalizam o que foi definido e analisado nas Seções 2.1 e 2.2 e as considerações de implementação da Seção 2.3; a correção da conversão para NNF e o limite inferior da conversão por distributividade encontram-se nas Proposições 2.1 e 2.2. Os métodos que a ferramenta não implementa, como a prenexação, a transformação de Tseitin e o método de Quine–McCluskey, permanecem descritos apenas no corpo do texto. Os scripts efetivamente utilizados na ferramenta constam do Apêndice B.

Nos algoritmos, uma *cláusula* (respectivamente, um *termo*) é representada como um conjunto de literais, e uma CNF (respectivamente, uma DNF) como um conjunto de cláusulas (respectivamente, de termos), isto é, na forma clausal discutida na Seção 2.3. O parâmetro L é o limite de tamanho da saída; quando excedido, o procedimento sinaliza o fato ao estudante em vez de prosseguir.

A.1 Conversão para a forma normal negativa

O Algoritmo 1 transcreve a função nnf da Seção 2.2, que carrega a polaridade da subfórmula corrente e elimina $\rightarrow$ e $\leftrightarrow$ na mesma passagem em que desloca as negações até os literais. A chamada inicial é $\textsc{Nnf}(\varphi, +)$. Como mostra a Proposição 2.1, o custo é linear em $|\varphi|$ na ausência de bicondicionais e pode ser exponencial na profundidade de seu aninhamento.

Algoritmo 1 Conversão para NNF com polaridade

Entrada: fórmula φ e polaridade $\pi \in \{+, -\}$
Saída: fórmula em NNF equivalente a φ (se $\pi = +$) ou a $\neg\varphi$ (se $\pi = -$)

1: **função** $\textsc{Nnf}(\varphi, \pi)$
2: **se** φ é uma variável p **então**
3: **devolver** p se $\pi = +$; $\neg p$ se $\pi = -$
4: **senão se** $\varphi = \neg\alpha$ **então**
5: **devolver** $\textsc{Nnf}(\alpha, \overline{\pi})$
6: **senão se** $\varphi = \alpha \wedge \beta$ **então**
7: **devolver** $\textsc{Nnf}(\alpha, \pi) \wedge \textsc{Nnf}(\beta, \pi)$ se $\pi = +$; $\textsc{Nnf}(\alpha, \pi) \vee \textsc{Nnf}(\beta, \pi)$ se $\pi = -$
8: **senão se** $\varphi = \alpha \vee \beta$ **então**
9: **devolver** $\textsc{Nnf}(\alpha, \pi) \vee \textsc{Nnf}(\beta, \pi)$ se $\pi = +$; $\textsc{Nnf}(\alpha, \pi) \wedge \textsc{Nnf}(\beta, \pi)$ se $\pi = -$
10: **senão se** $\varphi = \alpha \rightarrow \beta$ **então**
11: **devolver** $\textsc{Nnf}(\alpha, -) \vee \textsc{Nnf}(\beta, +)$ se $\pi = +$; $\textsc{Nnf}(\alpha, +) \wedge \textsc{Nnf}(\beta, -)$ se $\pi = -$
12: **senão se** $\varphi = \alpha \leftrightarrow \beta$ **então**
13: **se** $\pi = +$ **então**
14: **devolver** $\big(\textsc{Nnf}(\alpha, -) \vee \textsc{Nnf}(\beta, +)\big) \wedge \big(\textsc{Nnf}(\beta, -) \vee \textsc{Nnf}(\alpha, +)\big)$
15: **senão**
16: **devolver** $\big(\textsc{Nnf}(\alpha, +) \wedge \textsc{Nnf}(\beta, -)\big) \vee \big(\textsc{Nnf}(\beta, +) \wedge \textsc{Nnf}(\alpha, -)\big)$
17: **fim se**
18: **fim se**
19: **fim função**

A.2 Conversão para a forma normal conjuntiva e disjuntiva

A partir de uma fórmula em NNF, o Algoritmo 2 obtém a CNF por distributividade, tal como formulado na Seção 2.2: a conjunção corresponde à união dos conjuntos de cláusulas, e a disjunção, ao produto $\otimes$, que combina cada cláusula do primeiro conjunto com cada cláusula do segundo. É nesse produto que reside o custo exponencial, de modo que o limite L é verificado antes de calculá-lo.

Por dualidade, a DNF é obtida trocando-se os papéis da conjunção e da disjunção, como mostra o Algoritmo 3: a disjunção corresponde à união dos conjuntos de termos, e a conjunção, ao produto. O limite inferior da Proposição 2.2 vale, dualmente, para este caso.

A.3 Simplificações que preservam a equivalência

As simplificações opcionais mencionadas na Seção 2.3 operam sobre a forma clausal. O Algoritmo 4 trata o caso da CNF: elimina as cláusulas tautológicas, isto é, as que contêm um literal e seu complementar, e as cláusulas subsumidas, isto é, as que contêm propriamente outra cláusula do conjunto, pois $C \wedge (C \vee D) \equiv C$. O caso da DNF é dual:

Algoritmo 2 CNF por distributividade, a partir de uma NNF

Entrada: fórmula φ em NNF e limite L
Saída: conjunto de cláusulas equivalente a φ, ou sinalização de limite excedido

função $\textsc{Cnf}(\varphi, L)$
  se φ é um literal ℓ **então**
    devolver $\{\{\ell\}\}$
  senão se $\varphi = \alpha \wedge \beta$ **então**
    $A \leftarrow \textsc{Cnf}(\alpha, L)$; $B \leftarrow \textsc{Cnf}(\beta, L)$
    se $|A \cup B| > L$ **então**
      sinalizar LimiteExcedido
    fim se
    devolver $A \cup B$
  senão se $\varphi = \alpha \vee \beta$ **então**
    $A \leftarrow \textsc{Cnf}(\alpha, L)$; $B \leftarrow \textsc{Cnf}(\beta, L)$
    se $|A| \cdot |B| > L$ **então**
      sinalizar LimiteExcedido
    fim se
    devolver $\{\, C \cup D \mid C \in A,\ D \in B \,\}$
  fim se
fim função

Algoritmo 3 DNF por distributividade, a partir de uma NNF

Entrada: fórmula φ em NNF e limite L
Saída: conjunto de termos equivalente a φ, ou sinalização de limite excedido

função $\textsc{Dnf}(\varphi, L)$
  se φ é um literal ℓ **então**
    devolver $\{\{\ell\}\}$
  senão se $\varphi = \alpha \vee \beta$ **então**
    $A \leftarrow \textsc{Dnf}(\alpha, L)$; $B \leftarrow \textsc{Dnf}(\beta, L)$
    se $|A \cup B| > L$ **então**
      sinalizar LimiteExcedido
    fim se
    devolver $A \cup B$
  senão se $\varphi = \alpha \wedge \beta$ **então**
    $A \leftarrow \textsc{Dnf}(\alpha, L)$; $B \leftarrow \textsc{Dnf}(\beta, L)$
    se $|A| \cdot |B| > L$ **então**
      sinalizar LimiteExcedido
    fim se
    devolver $\{\, C \cup D \mid C \in A,\ D \in B \,\}$
  fim se
fim função

eliminam-se os termos contraditórios e os termos que contêm propriamente outro termo do conjunto. No pior caso, o procedimento compara todos os pares de cláusulas, de modo que seu custo é quadrático no tamanho do conjunto.

Algoritmo 4 Simplificação de uma CNF

Entrada: conjunto de cláusulas Γ
Saída: conjunto de cláusulas equivalente a Γ, sem tautologias e sem subsunção

função SIMPLIFICAR(Γ)
    $\Gamma \leftarrow \{\, C \in \Gamma \mid C$ não contém um literal e seu complementar $\}$
    devolver $\{\, C \in \Gamma \mid$ não existe $D \in \Gamma$ com $D \subsetneq C \,\}$
fim função

A.4 Procedimento completo

O Algoritmo 5 compõe os anteriores no procedimento que a ferramenta executa: a conversão para NNF, seguida da conversão para a forma-alvo e, se solicitado, da simplificação.

Algoritmo 5 Conversão completa para CNF ou DNF

Entrada: fórmula φ, forma-alvo $\in \{\mathrm{CNF}, \mathrm{DNF}\}$, limite L, indicador de simplificação
Saída: forma normal equivalente a φ, ou sinalização de limite excedido

função CONVERTER(φ, alvo, L, simplificar)
    $\psi \leftarrow \mathrm{NNF}(\varphi, +)$
    se alvo = CNF **então**
        $R \leftarrow \mathrm{CNF}(\psi, L)$
    senão
        $R \leftarrow \mathrm{DNF}(\psi, L)$
    fim se
    se simplificar **então**
        $R \leftarrow$ SIMPLIFICAR(R) ▷ na versão dual, se alvo = DNF
    fim se
    devolver R
fim função

Apêndice B

Scripts de Transformação

B.1 Scripts de Transformação